

CoachCalories AI

Progetto di Applicazioni e Servizi Web

Federico Bonetti - 1132808
federico.bonetti2@studio.unibo.it

29 maggio 2026

1 Introduzione

1.1 L'importanza del bilancio calorico

Raggiungere specifici obiettivi fisici, che si tratti di dimagrimento, ipertrofia muscolare o semplice ricomposizione corporea, richiede un approccio analitico all'alimentazione. Il principio termodinamico fondamentale alla base della variazione di peso è il bilancio energetico, spesso calcolato tramite il TDEE (Total Daily Energy Expenditure). Operare in deficit o in surplus calorico rispetto a tale soglia determina, rispettivamente, la perdita o l'aumento di peso.

Tuttavia, il solo conteggio calorico risulta riduttivo se non accompagnato da un corretto bilanciamento dei macronutrienti. Le proteine sono essenziali per il mantenimento e la sintesi del tessuto muscolare; i carboidrati rappresentano il substrato energetico primario per le prestazioni fisiche e cognitive, mentre i grassi sono fondamentali per la regolazione del sistema ormonale. Nonostante la comprovata importanza del monitoraggio nutrizionale (tracking), i metodi tradizionali forniti dalle applicazioni classiche risultano spesso macchinosi. Dover cercare ogni singolo alimento, calcolare a mente le proporzioni in base ai grammi e inserire i dati manualmente genera un forte "attrito" (user friction), portando a un alto tasso di abbandono della pratica.

1.2 Il calcolo del TDEE (Total Daily Energy Expenditure)

Il TDEE rappresenta la stima del dispendio energetico totale giornaliero di un individuo. È il valore termodinamico fondamentale su cui si basa l'algoritmo dell'applicazione per valutare l'andamento del diario alimentare dell'utente. Il TDEE è composto dalla somma di diversi fattori metabolici: il metabolismo basale (BMR, l'energia necessaria per il mantenimento delle funzioni vitali), la termogenesi indotta dalla dieta (TEF) e il dispendio energetico derivante dall'attività fisica (sia l'allenamento programmato che la termogenesi da attività non associabile all'esercizio, nota come NEAT).

A livello computazionale, il sistema ricava innanzitutto il metabolismo basale applicando modelli matematici validati scientificamente, come l'equazione di Mifflin-St Jeor:

$$BMR = (10 \times peso) + (6.25 \times altezza) - (5 \times età) + s$$

(dove la costante $s = +5$ per il sesso maschile e $s = -161$ per quello femminile).

Il valore base ottenuto viene poi elaborato moltiplicandolo per uno specifico coefficiente di attività ($C_{attività}$), che parametrizza lo stile di vita dell'utente (da sedentario a estremamente attivo), calcolando così il dispendio totale:

$$TDEE = BMR \times C_{attività}$$

1.3 I tre scenari metabolici: Mantenimento, Dimagrimento e Ipertrofia

Una volta calcolato il TDEE, l'applicazione parametrizza i feedback e i target nutrizionali in base allo scopo specifico impostato dall'utente. Questo si declina in tre precisi scenari termodinamici:

- **Mantenimento (Normocalorica):** L'obiettivo è mantenere invariato il peso corporeo attuale. In questo caso, l'algoritmo imposta un apporto calorico target che coincide esattamente con il dispendio energetico ($Calorie = TDEE$). Il focus del sistema di monitoraggio si sposta dal controllo calorico stringente al bilanciamento dei macronutrienti, per garantire il mantenimento dell'omeostasi e la salute generale dell'utente.
- **Dimagrimento (Deficit Calorico):** L'obiettivo è la riduzione della massa grassa (o *cutting*). Il sistema elabora un target calorico inferiore al TDEE ($Calorie < TDEE$), applicando solitamente una decurtazione di circa 300-500 kcal per indurre il corpo a intaccare le riserve adipose. In questo scenario, assume un'importanza cruciale il monitoraggio dell'apporto proteico per preservare la massa magra durante il deperimento energetico.
- **Aumento di Massa (Surplus Calorico):** L'obiettivo è l'ipertrofia, ovvero la costruzione di nuovo tessuto muscolare (o *bulking*). Affinché il corpo disponga dell'energia necessaria per sintetizzare nuove proteine, è richiesto un ambiente anabolico garantito da un apporto calorico superiore al dispendio ($Calorie > TDEE$). L'applicazione suggerisce un surplus calorico controllato per minimizzare l'accumulo di massa grassa, supportato da un adeguato introito di carboidrati per massimizzare la resa nelle prestazioni fisiche.

1.4 Obiettivi del progetto: CoachCalories AI

Il progetto *CoachCalories AI* si propone come una piattaforma web dinamica e modulare, sviluppata con l'obiettivo primario di ingegnerizzare e semplificare il

processo di tracciamento alimentare quotidiano. L'applicazione mira a facilitare la registrazione dei pasti attraverso interfacce utente intuitive e responsive, strutturate per elaborare e salvare in modo automatizzato l'apporto calorico e lo spettro dei macronutrienti dei cibi consumati.

Sulla base dei dati storici memorizzati, il sistema genera grafici e statistiche sull'andamento del bilancio energetico, offrendo all'utente un quadro analitico visivo dei propri progressi. All'interno di questa architettura, il calcolo del TDEE funge da metrica di riferimento dinamica, venendo computato e aggiornato dal sistema sulla base dei parametri fisici e antropometrici inseriti dall'utente nel proprio profilo.

Per garantire l'affidabilità scientifica del sistema e l'integrità dei dati, l'applicazione implementa una rigorosa logica di controllo degli accessi basata sui ruoli (Role-Based Access Control). La popolazione del database globale, con l'aggiunta di nuovi alimenti e la certificazione dei relativi macro e calorie, è interamente preclusa all'utente comune e consentita esclusivamente a uno staff scelto di amministratori (Admin). Questo assicura che il cuore informativo del sistema rimanga protetto da inserimenti errati o ridondanti.

Infine, come funzionalità aggiuntiva per arricchire l'interazione e abbattere l'attrito tipico del monitoraggio manuale, l'infrastruttura web integra una componente di Intelligenza Artificiale. Sfruttando le API di un Large Language Model, questa estensione permette all'utente di interagire con l'applicazione tramite linguaggio naturale (es. inserendo interi pasti descritti a testo libero), delegando all'algoritmo l'estrazione delle entità e la quantificazione dei macro. L'IA funge quindi da assistente complementare, capace anche di fornire consigli contestuali basati sullo stato attuale del diario, migliorando sensibilmente la User Experience complessiva.

2 Requisiti

In questa sezione vengono elencati i requisiti del sistema *CoachCalories AI*. I requisiti sono divisi in Funzionali (funzionalità offerte dal sistema) e Non Funzionali (vincoli tecnologici e prestazionali), identificati rispettivamente dalle sigle **RF** e **RNF**. Il sistema adotta un controllo degli accessi basato su tre ruoli: ospite, utente registrato e amministratore.

2.1 Requisiti Funzionali

- **RF1 - Autenticazione:** Il sistema deve consentire l'accesso agli utenti nel sistema tramite login con email e password.
- **RF2 - Gestione Logout:** Il sistema deve distruggere la sessione corrente al click sul pulsante di uscita.
- **RF3 - Inserimento Dati Fisiologici:** Il sistema deve permettere l'inserimento di peso, altezza, età, sesso e attività fisica.

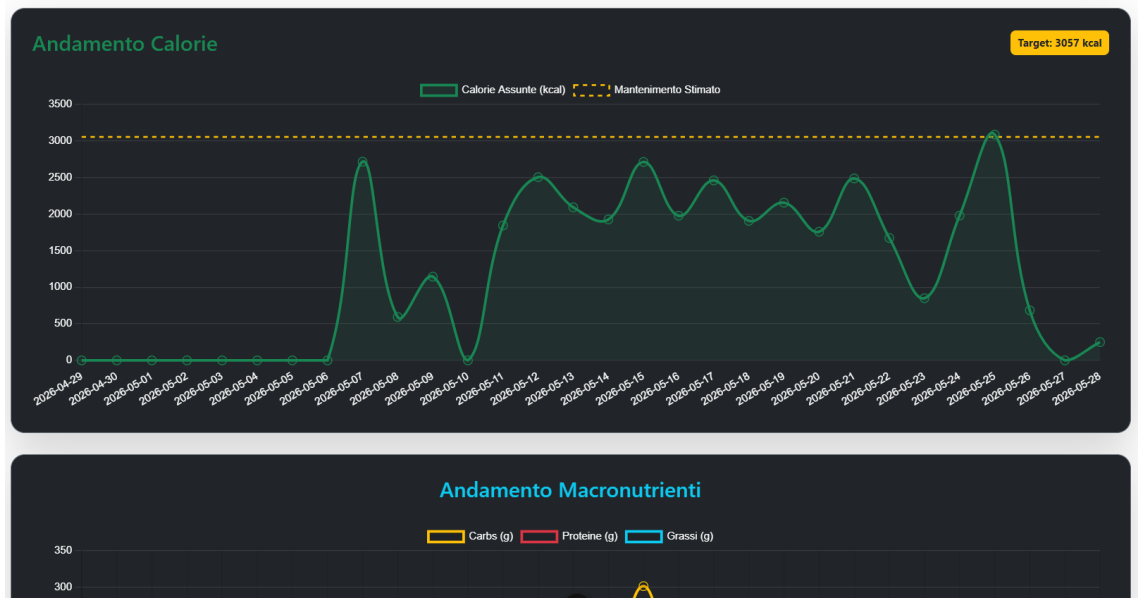


Figura 1: Esempio di un grafico personalizzabile nel sistema

- **RF4 - Calcolo TDEE:** L'applicazione deve calcolare il fabbisogno calorico giornaliero dell'utente incrociando tutti i parametri fisici inseriti con il livello di attività selezionato.
- **RF5 - Storizzazione Dati:** Il sistema deve archiviare automaticamente i vecchi parametri fisici a ogni nuova modifica per preservare i dati temporali passati.
- **RF6 - Visualizzazione Storico:** Il sistema deve mostrare una tabella cronologica con tutte le variazioni fisiche passate dell'utente.
- **RF7 - Eliminazione Storico:** L'utente deve poter cancellare i singoli record archiviati tramite un pulsante dedicato senza intaccare i dati correnti.
- **RF8 - Tracciamento Diario:** Il sistema deve permettere l'aggiunta di alimenti e relative grammature nel diario giornaliero delle calorie.
- **RF9 - Calcolo Macronutrienti:** L'applicazione deve sommare in tempo reale calorie, carboidrati, proteine e grassi assunti durante la giornata.
- **RF10 - Grafici Dinamici:** Il frontend deve mostrare grafici interattivi per confrontare i macro assunti con il target energetico giornaliero calcolato.

- **RF11 - Interazione Assistente IA:** Il sistema deve integrare una chat per comunicare in linguaggio naturale con un Coach virtuale di supporto.
- **RF12 - Compilazione Automatica Diario:** L'assistente IA deve saper interpretare i messaggi ed eseguire chiamate a funzioni per inserire i cibi nel diario in modo autonomo.
- **RF13 - Invio Proposte Alimenti:** Gli utenti non admin devono poter proporre nuovi cibi descrivendone i valori nutrizionali per l'inserimento nel catalogo.
- **RF14 - Approvazione Proposte:** L'amministratore deve poter accettare o rifiutare le proposte inviate dagli utenti per validare la qualità dei dati.
- **RF15 - Gestione Catalogo Admin:** L'amministratore deve poter inserire, modificare o eliminare direttamente gli alimenti dal catalogo globale.
- **RF16 - Consultazione Catalogo:** Tutti gli utenti, inclusi gli ospiti, devono poter visualizzare la lista complessiva degli alimenti del database.
- **RF17 - Centro Notifiche:** Il sistema deve mostrare avvisi e aggiornamenti all'utente all'interno di un menu dedicato per segnalare le azioni principali.

2.2 Requisiti Non Funzionali

- **RNF1 - Stack Tecnologico:** Il sistema deve essere sviluppato interamente utilizzando lo Stack MEVN (MongoDB, Express, Vue, Node).
- **RNF2 - Latenza dell'Intelligenza Artificiale:** Le risposte della chat del Coach IA devono avvenire con tempi di attesa minimi sfruttando l'infrastruttura Groq.
- **RNF3 - Protocollo di Rete Locale:** L'applicazione accetta connessioni sul protocollo standard HTTP durante la fase di sviluppo in locale.

3 Design

L'applicazione è sviluppata seguendo il paradigma delle *Single Page Application* (SPA), il che significa che la navigazione tra le diverse sezioni non comporta il ricaricamento effettivo della pagina nel browser, ma viene gestita dinamicamente attraverso la manipolazione dello stato reattivo dei componenti.

L'elemento cardine di questa architettura è la barra di navigazione (*navbar*), il cui aspetto visivo e le cui voci di menu dipendono strettamente dal livello di autenticazione dell'utente corrente. Il sistema implementa infatti un controllo degli accessi basato sui ruoli (Role-Based Access Control - RBAC), che adatta l'interfaccia e ne seziona le funzionalità in tre scenari principali: utente ospite, utente registrato e amministratore.

3.1 Interfaccia per l'Utente Ospite (Non Autenticato)

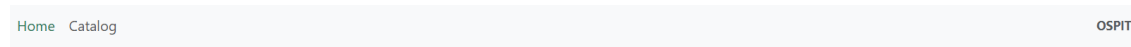


Figura 2: La navbar nel caso l'utente sia ospite

L'utente ospite rappresenta chiunque acceda alla piattaforma senza aver effettuato l'autenticazione nel sistema. Per ragioni di sicurezza e di usabilità, l'interfaccia per questo ruolo si presenta in una modalità minimale e fortemente limitata, esponendo solo le funzionalità di consultazione pubblica e i punti di accesso.

Nel dettaglio, il design per l'utente ospite prevede la seguente disposizione:

- **Barra di Navigazione Sfoltita:** La *navbar* superiore mostra unicamente i collegamenti alle pagine *Home* e *Catalog*. Sulla parte destra della barra, un'etichetta testuale mostra esplicitamente lo stato di "*Ospite*", segnalando visivamente che non è attiva alcuna sessione di profilo. In questa modalità non compare alcun pulsante di disconnessione (Logout) né il menu a tendina per le notifiche.
- **Home Page e Form di Accesso:** La schermata principale di atterraggio funge da hub di accoglienza e barriera di sicurezza. Al centro della pagina è posizionato il modulo di login strutturato con due campi di input ('Email' e 'Password'). Fino a quando l'utente non inserisce credenziali valide riconosciute dal backend, la navigazione verso il cuore operativo dell'applicazione rimane bloccata.
- **Catalogo Alimentare Pubblico:** L'unica reale schermata operativa accessibile all'ospite è la pagina *Catalog*. All'interno di questa sezione, l'utente non autenticato può consultare liberamente la lista complessiva di tutti gli alimenti censiti nel database, visionandone i relativi valori macro-nutrizionali (calorie, carboidrati, proteine e grassi). Tuttavia, l'interfaccia non permette alcuna azione attiva: l'ospite può solo leggere i dati, senza poter aggiungere elementi al diario o inviare nuove proposte di cibo.

Tutte le viste avanzate dell'applicazione — come il Diario Alimentare, i Grafici di Analisi, il Profilo Fisico e la chat con il Coach IA — sono completamente rimosse dal DOM del browser tramite direttive condizionali di Vue, rendendo impossibile per l'ospite anche solo vederne la presenza nel menu.

3.2 Interfaccia per l'Utente Registrato (Standard)

Una volta eseguito il login con credenziali valide, l'interfaccia utente si sblocca completamente, rivelando l'intera mappa dell'applicazione. La barra di navigazione si aggiorna dinamicamente: scompare il form di login e compaiono i

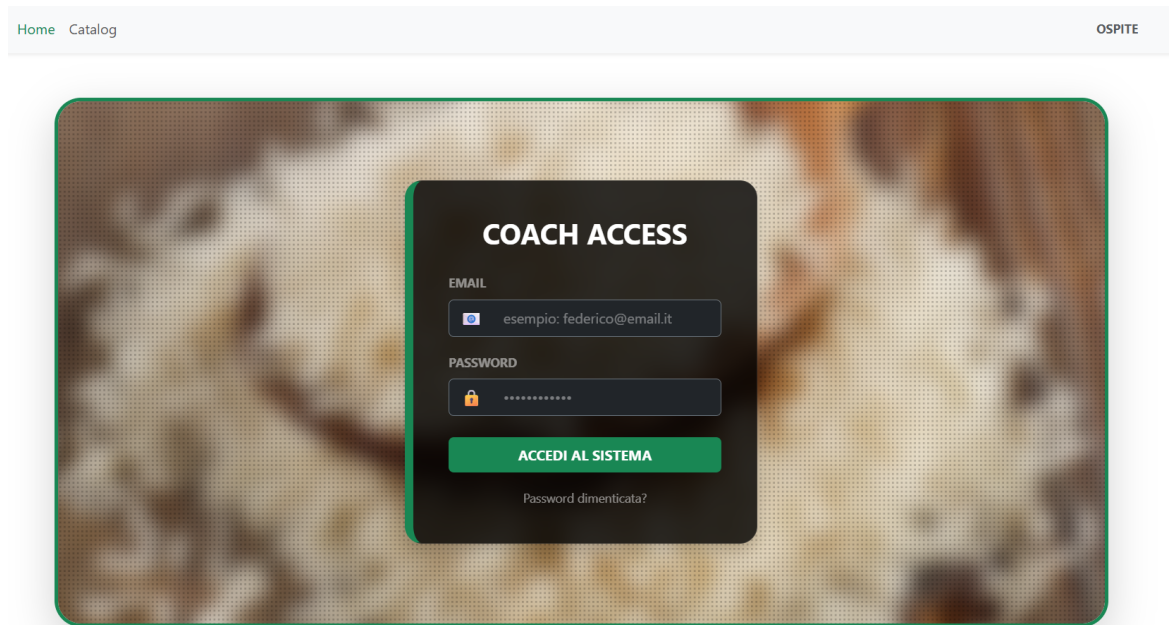


Figura 3: Pagina di login

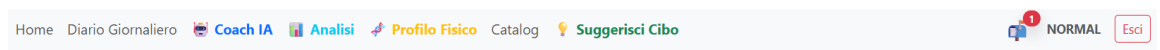


Figura 4: La navbar nel caso l'utente sia autenticato

collegamenti rapidi a tutte le aree di tracciamento e analisi, oltre al menu per la gestione delle notifiche personali e al pulsante di disconnessione (*Logout*).

Il design operativo per l'utente standard si articola nelle seguenti sezioni e pagine dedicate:

- **Home Page (Dashboard):** Dopo il login, la schermata principale si trasforma da semplice form di accesso a una *dashboard* di benvenuto. La pagina mostra un messaggio di accoglienza dinamico con il nome utente e offre una panoramica immediata dello stato del profilo (come la data odierna e un promemoria rapida sui dati fisiologici correnti), fungendo da hub centrale con scorciatoie visive per saltare rapidamente alle sezioni operative come il diario o la chat con il Coach.
- **Diario Alimentare Giornaliero:** Rappresenta la schermata principale per il monitoraggio quotidiano. L'interfaccia visualizza un riepilogo in tempo reale dell'apporto calorico odierno, suddiviso per i pasti principali della giornata (Colazione, Pranzo, Cena, Snack). L'utente dispone di un sistema di inserimento rapido che permette di cercare gli alimenti all'in-

terno del catalogo globale e specificarne la grammatura consumata per aggiornare istantaneamente il diario.

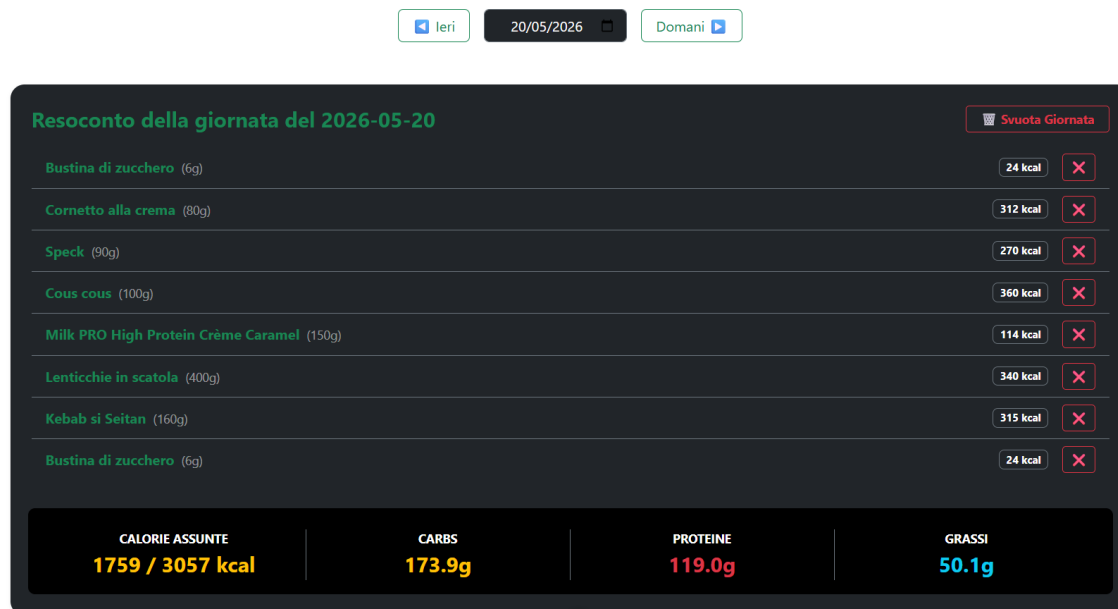


Figura 5: Pagina del diario giornaliero (parte alta)

- **Analisi Nutrizionale e Grafici:** Questa pagina è deputata alla visualizzazione statistica dei dati. Attraverso l'uso di grafici interattivi (renderizzati tramite la libreria *Chart.js*), l'interfaccia mostra all'utente la scomposizione percentuale dei macronutrienti assunti (carboidrati, proteine e grassi) rispetto al totale giornaliero. Inoltre, un indicatore grafico confronta visivamente la quota calorica introdotta con il target energetico ideale calcolato, permettendo all'utente di capire a colpo d'occhio se si trova in deficit, surplus o mantenimento calorico.
- **Profilo Fisico e Centro Metabolico:** È la sezione dedicata alla gestione dei parametri antropometrici dell'utente. A sinistra, la pagina presenta un modulo per l'immissione e la modifica dei propri dati (Peso, Altezza, Età, Sesso e Livello di attività fisica). A destra, un pannello reattivo mostra immediatamente il calcolo del Dispendio Energetico Giornaliero (TDEE). Nella parte inferiore è collocata la tabella dello *Storico Variazioni*: grazie alla logica di storicizzazione del database, l'interfaccia mostra l'evoluzione temporale dei parametri fisici dell'utente. I vecchi record vengono visualizzati con un badge di stato "Archiviato" e una data di fine validità, e sono affiancati da un pulsante a forma di cestino che ne consente l'eliminazione.

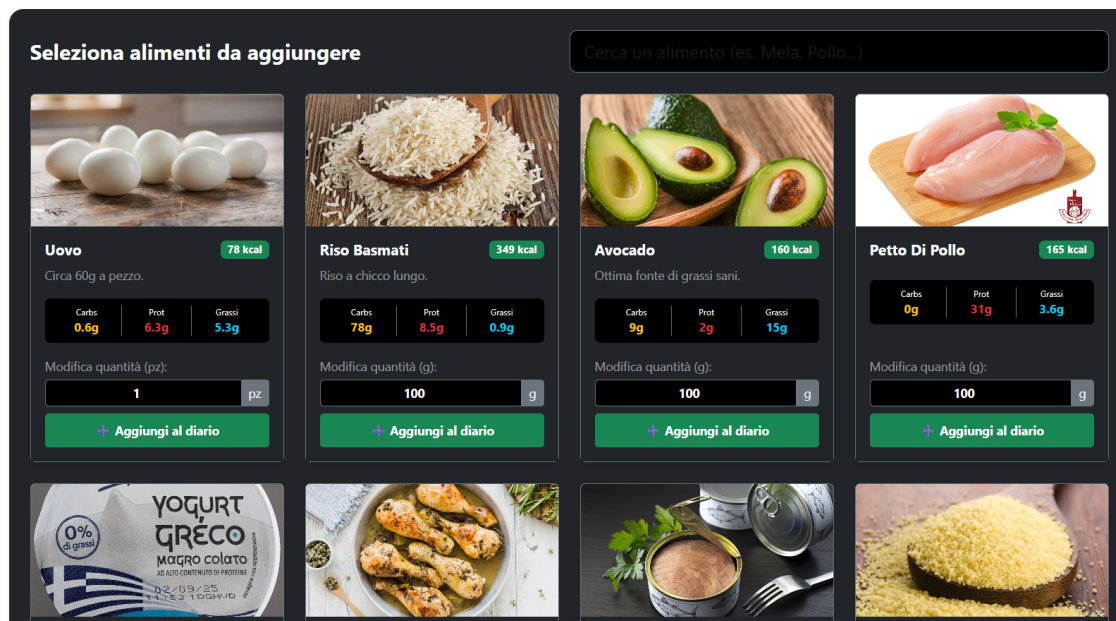


Figura 6: Pagina del diario giornaliero (parte bassa)

definitiva. Il record corrente viene invece contrassegnato dal badge verde "Attivo" ed è protetto contro la cancellazione accidentale.

- Coach Virtuale IA:** Questa schermata implementa l'interfaccia di messaggistica con l'assistente artificiale. Il design simula una moderna applicazione di chat, in cui l'utente può conversare liberamente in linguaggio naturale per descrivere i pasti consumati (es. *"Oggi a pranzo ho mangiato 80 grammi di pasta al pomodoro e un cucchiaino di olio"*). L'interfaccia mostra lo streaming di risposta del modello linguistico e aggiorna in background la schermata del diario alimentare, sfruttando la capacità dell'IA di interpretare il testo ed eseguire l'aggiornamento automatico dei pasti senza che l'utente debba compilare manualmente alcun modulo.
- Suggerisci Alimento:** Una pagina di interazione con la comunità in cui l'utente standard può contribuire all'espansione dell'applicazione. Attraverso un form guidato, è possibile proporre l'inserimento di un cibo non ancora presente nel sistema, specificandone il nome, la marca e la tabella nutrizionale per 100 grammi di prodotto. Una volta inviato, il sistema notifica l'utente che la proposta è in stato di attesa e passerà al vaglio dell'amministratore per la validazione.

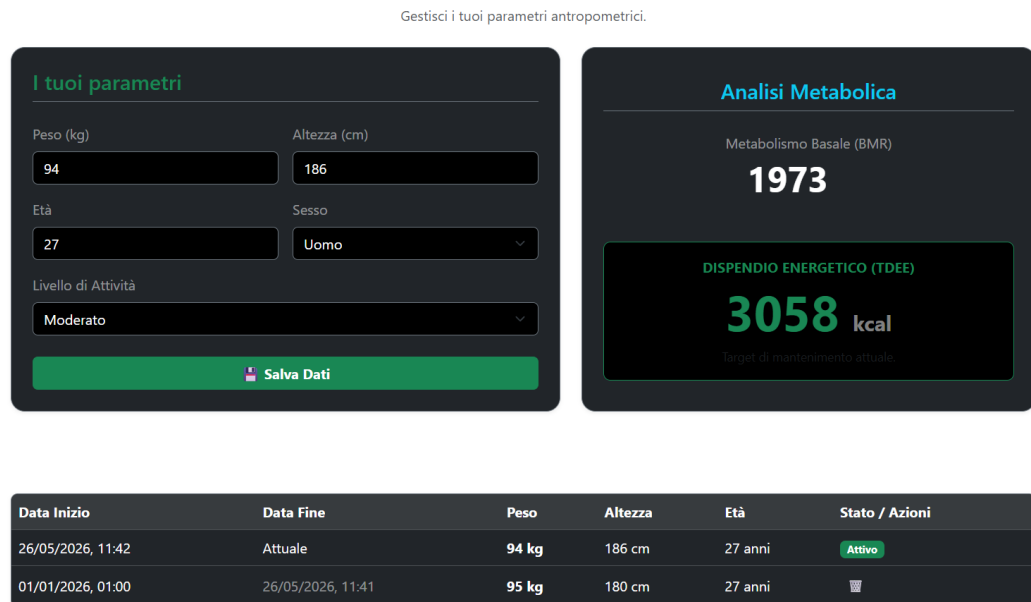


Figura 7: Pagina del diario giornaliero (parte bassa)

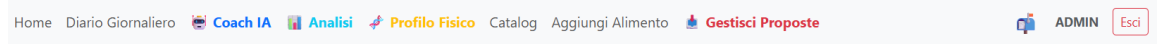


Figura 8: La navbar nel caso l'utente sia admin

3.3 Interfaccia per l'Amministratore (Admin)

L'utente con privilegi di amministratore (*Admin*) riveste un doppio ruolo all'interno della piattaforma. Oltre a disporre degli strumenti esclusivi per la supervisione dei dati, **l'amministratore mantiene l'accesso completo a tutte le funzionalità operative dell'utente standard**. Questo gli consente di utilizzare l'applicazione in prima persona per registrare i propri pasti quotidiani nel diario, tracciare l'apporto dei macronutrienti, consultare i grafici di analisi, interagire con il Coach IA e configurare il proprio profilo fisico per seguire i propri piani alimentari basati sul TDEE.

L'unica eccezione rispetto all'interfaccia utente base riguarda la pagina di *Suggerimento Alimento*, la quale viene volutamente nascosta al ruolo Admin in quanto ridondante: l'amministratore ha infatti il potere di inserire direttamente i cibi nel database globale senza dover passare da una fase di sottomissione e approvazione.

A livello di design, la barra di navigazione dell'Admin si arricchisce integrando, oltre alle normali viste di tracciamento personale, i collegamenti rapidi ai pannelli di controllo centralizzati, i quali si articolano nelle seguenti aree esclusive:

Calorie Coach

Gestisci la tua alimentazione con precisione.

Cerca un alimento (es. Pollo, Avocado...)

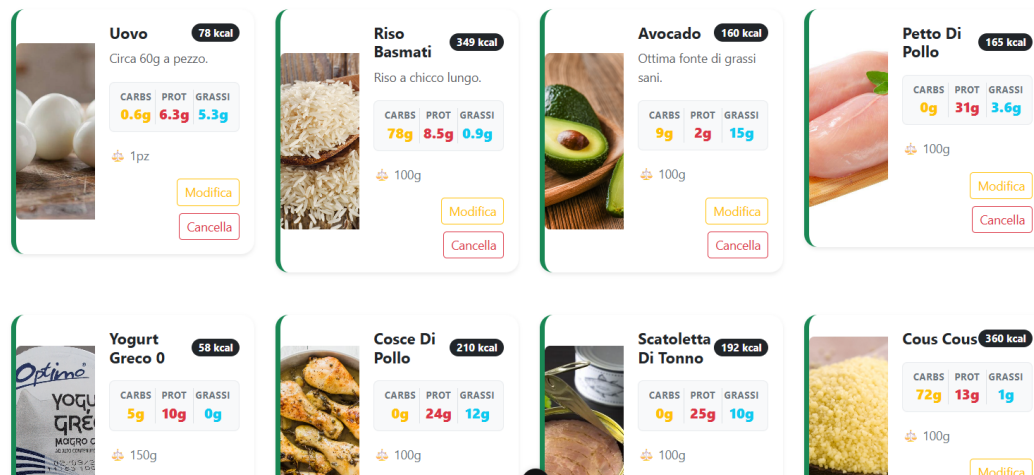


Figura 9: Pagina del catalogo vista da lato admin

- **Pannello di Gestione del Catalogo Globale:** Questa schermata si presenta come una potente *dashboard* di controllo per l'intero database alimentare dell'applicazione. L'interfaccia mostra una tabella riepilogativa di tutti i cibi censiti nel sistema. Per ogni alimento sono presenti due azioni dirette: un pulsante di modifica (*Edit*) — che apre un modulo precompilato per correggere al volo i valori nutrizionali o il nome del prodotto — e un pulsante di eliminazione (*Delete*) per rimuovere il cibo dal catalogo. In cima alla pagina, un pulsante di inserimento consente l'apertura di un form per l'aggiunta di un nuovo alimento da zero. Questo form gestisce il caricamento dei file multimediali, interfacciandosi con la logica del server per l'invio e il salvataggio dell'immagine identificativa del cibo.
- **Pagina “Aggiungi Alimento”:** Questa schermata rappresenta il punto di inserimento diretto di nuove risorse nel database, accessibile esclusivamente all'utente amministratore. L'interfaccia è progettata per garantire un data-entry rapido e privo di errori, articolandosi in tre macro-aree:
 - *Dati Anagrafici:* Campi di testo per specificare il nome dell'alimento e la marca o produttore, utili a evitare ambiguità nel catalogo.
 - *Tabella Nutrizionale (per 100g):* Una serie di moduli di input numerici in cui l'amministratore deve obbligatoriamente inserire l'apporto calorico complessivo (in kcal) e la ripartizione dei tre macronu-

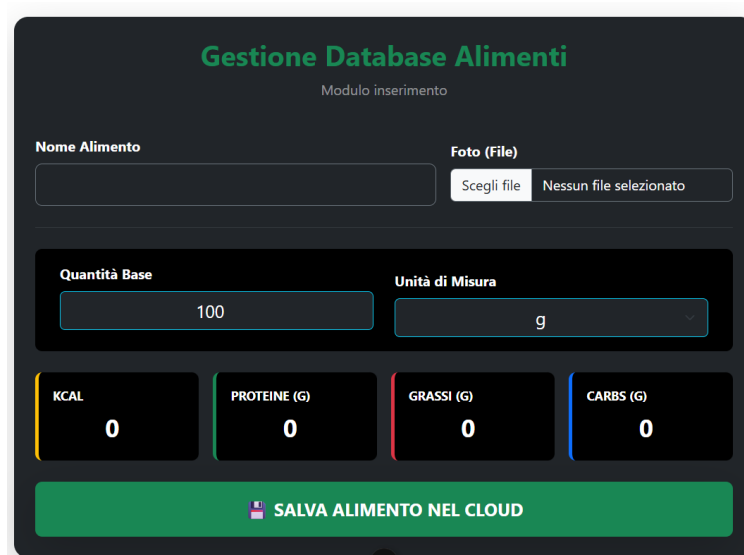


Figura 10: Pagina per aggiungere un alimento

trienti fondamentali, ovvero carboidrati, proteine e grassi (espressi in grammi).

- *Caricamento Elementi Multimediali:* Un componente di selezione file (*file picker*) che permette di allegare un'immagine rappresentativa del cibo. Questa sezione si interfaccia direttamente con il backend per gestire il flusso binario del file tramite il middleware *Multer*.

La pagina integra un sistema di validazione reattivo che disabilita il pulsante di invio se i campi obbligatori sono incompleti o se i valori nutrizionali inseriti non rispecchiano i vincoli logici (es. valori negativi), mostrando messaggi di successo o di errore tramite alert temporanei a caricamento completato.

- **Hub di Approvazione delle Proposte Utente:** È lo spazio dedicato alla moderazione delle richieste inviate dagli utenti standard tramite la pagina di suggerimento cibi. L'interfaccia visualizza un elenco di schede (*cards*) corrispondenti alle proposte ancora in stato di attesa (*pending*). Ogni scheda mostra chiaramente i dettagli inseriti dall'utente (Nome cibo, marca, calorie e ripartizione dei tre macronutrienti). Al fondo di ciascuna proposta sono posizionati due controlli decisionali netti: il pulsante "Approva" (contrassegnato graficamente in verde), che convalida il cibo inserendolo istantaneamente nel catalogo globale a disposizione di tutti, e il pulsante "Rifiuta" (in rosso), che scarta la proposta rimuovendola dalla coda di moderazione.

Approvazione Proposte

[Torna al Catalogo](#)

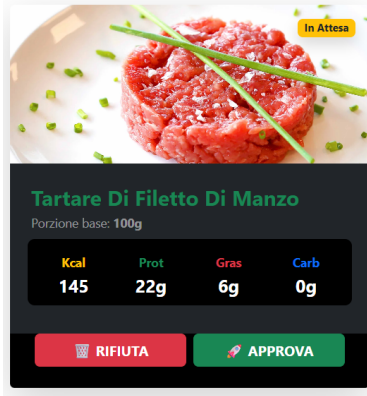


Figura 11: Pagina per visionare le proposte degli utenti

Attraverso questa netta separazione delle interfacce, l'amministratore è in grado di operare sul database in modo sicuro e centralizzato, godendo di una panoramica chiara che azzerà il rischio di inserimenti duplicati o errati.

4 Tecnologie

L'infrastruttura portante di *CoachCalories AI* si basa sul consolidato **Stack MEVN** (MongoDB, Express.js, Vue.js, Node.js). Trattandosi di un ecosistema Full-Stack interamente basato su JavaScript standard per lo sviluppo web moderno, la sua implementazione garantisce una gestione fluida delle richieste asincrone, accoppiando un database NoSQL flessibile a documenti (MongoDB) con un backend reattivo (Node/Express) e un frontend modulare guidato da componenti dinamici (Vue.js).

Il vero valore ingegneristico dell'applicazione risiede tuttavia nell'integrazione di moduli e servizi verticali capaci di trasformare la piattaforma da una semplice architettura CRUD a un sistema analitico e interattivo avanzato. Di seguito vengono analizzate nel dettaglio le tecnologie chiave che gestiscono la visualizzazione dei dati, i flussi multimediali e l'elaborazione semantica:

- **Chart.js (Visualizzazione Analitica e Reattività dei Dati):** Libreria open-source basata su HTML5 Canvas integrata lato client per la generazione di grafici interattivi. Invece di limitarsi a mostrare record testuali grezzi, il frontend sfrutta Chart.js per elaborare i dati storici aggregati provenienti da MongoDB. Il sistema mappa i macronutrienti giornalieri (carboidrati, proteine, grassi) e l'andamento del bilancio energetico rispetto al TDEE calcolato dell'utente, eseguendo rendering dinamici e re-

sponsive che si aggiornano istantaneamente a ogni inserimento o rimozione nel diario. Questa tecnologia si rivela cruciale per tradurre metriche numeriche complesse in feedback visivi immediati e facilmente interpretabili dall'utente.

- **Multer (Ingegnerizzazione dei Flussi Multipart/Form-Data):** Middleware fondamentale per Node.js deputato alla gestione e al salvataggio dei file sul server (*file upload*). Poiché il framework Express non è strutturato per decodificare nativamente i payload multimediali, l'integrazione di Multer è stata indispensabile per gestire richieste di tipo *multipart/form-data* in due scenari di business essenziali: il caricamento di immagini di copertina da parte degli Admin nel catalogo cibi ufficiale e l'invio di foto allegate da parte degli utenti alle proposte di nuovi alimenti (*Food Proposals*). Il modulo intercetta i file, ne valida il formato, previene collisioni di naming sovrascrivendo i timestamp e organizza fisicamente gli asset nella directory locale `public/img/foods/`, registrando poi nel database la stringa del percorso univoco.
- **Groq API & Llama 3.3 70B Versatile (Infrastruttura di Inferenza NLP e Tool Calling):** Engine computazionale esterno integrato per abbattere l'attrito del tracciamento manuale attraverso l'elaborazione del linguaggio naturale. Sfruttando l'infrastruttura hardware LPU (Language Processing Unit) di Groq, l'applicazione garantisce una velocità di inferenza estremamente elevata con una latenza vicina allo zero, elemento fondamentale per mantenere la UX della chat fluida. Il modello Llama 3.3 (da 70 miliardi di parametri) viene istruito per eseguire operazioni di *structured output* e *tool calling*. L'algoritmo non si limita a rispondere testualmente, ma analizza la frase a testo libero dell'utente, isola le entità (nome cibo e grammi), stima i macro mancanti e mappa la richiesta eseguendo chiamate a funzioni specifiche del backend (`inserisci_cibo_oggi`, `autocompila_giorno_vuoto`), compilando in autonomia i payload JSON necessari all'aggiornamento del diario.

5 Codice (Aspetti Rilevanti)

L'architettura software di *CoachCalories* si fonda su una netta separazione tra il livello logico-funzionale e di persistenza dei dati (*Backend*) e il livello di presentazione e interazione utente (*Frontend*). Coerentemente con i vincoli imposti dallo sviluppo di una **Single Page Application (SPA)**, sul versante client l'applicazione implementa il pattern architetturale **MVVM (Model-View-ViewModel)** nativo di Vue.js. In questo ecosistema, il template HTML5 agisce come *View*, mentre i blocchi logici basati sulla *Composition API* fungono da *ViewModel*, orchestrando lo stato reattivo e sincronizzandolo biunivocamente con l'interfaccia grafica senza richiedere manipolazioni dirette del DOM.

Per illustrare l'ingegnerizzazione e l'organizzazione modulare del codice, di seguito vengono analizzati i flussi di navigazione interna e la scomposizione

strutturale delle schermate, differenziando i componenti in base alla loro densità dinamica e al rispettivo ciclo di vita.

5.1 Frontend

L'intera interfaccia utente e la logica di presentazione applicativa sono centralizzate all'interno del client Vue.js. L'architettura del frontend è strutturata per scomporre l'applicazione in componenti riutilizzabili, isolando la logica di stato e ottimizzando i cicli di rendering sul browser attraverso le seguenti implementazioni specifiche.

5.1.1 Orchestrazione della Sessione e Navigazione di Stato (App.vue e NavBar.vue)

Il ciclo di vita e lo stato globale del client sono interamente governati dal componente radice **App.vue**. Al fine di ottimizzare le prestazioni complessive e mantenere l'applicazione snella, la transizione tra le diverse schermate non si affida a un sistema di routing di terze parti, bensì a un meccanismo proprietario di **Routing di Stato** guidato da variabili reattive e flussi di persistenza locale.

All'atto del caricamento o a seguito di un ripristino della pagina (*browser refresh*), il ciclo di inizializzazione sincrono di **App.vue** intercetta i dati di autenticazione memorizzati nel **localStorage**. Questo approccio previene il fenomeno del *flickering* grafico e valida istantaneamente l'autorizzazione dell'utente prima del rendering dei componenti figli:

```
1 // App.vue (<script setup>)
2 import { ref } from 'vue'
3 import NavBar from './components/NavBar.vue'
4 import HomeContainer from './components/HomeContainer.vue'
5 import DailyDiaryPage from './pages/DailyDiaryPage.vue'
6
7 const isLoggedIn = ref(false)
8 const userGrade = ref('')
9 const currentPage = ref('Home')
10
11 // Ripristino immediato dello stato della sessione prima del
    mount grafico
12 const savedEmail = localStorage.getItem('userEmail')
13 const savedGrade = localStorage.getItem('authGrade')
14 if (savedEmail) {
15   isLoggedIn.value = true
16   userGrade.value = savedGrade
17
18   // Recupero della pagina attiva per preservare la
    cronologia utente
19   const savedPage = localStorage.getItem('currentPage')
20   if (savedPage) currentPage.value = savedPage
21 }
```

```

22
23 const setPage = (pageName) => {
24   currentPage.value = pageName
25   localStorage.setItem('currentPage', pageName)
26 }

```

Listing 1: Estrazione della sessione e navigazione reattiva nel componente radice App.vue.

Il disaccoppiamento tra la logica di controllo e gli elementi di interfaccia è garantito da un pattern di comunicazione ad eventi (*Event-Driven Architecture* locale). Il componente `NavBar.vue` riceve in ingresso il grado di autorizzazione dell'utente (`userGrade`) sotto forma di *Prop* monodirezionale e, tramite la macro `defineEmits`, espone al componente padre i segnali di transizione.

All'interno del template di `NavBar.vue`, i link di navigazione intercettano l'evento nativo di click, ne inibiscono il comportamento di ricarica predefinito tramite il modificatore `.prevent` ed emettono il segnale `navigate` associato alla stringa identificativa della destinazione:

```

1 <script setup>
2 const props = defineProps(['userGrade']);
3 const emit = defineEmits(['logout', 'navigate']);
4 </script>
5
6 <template>
7   <ul class="navbar-nav me-auto mb-2 mb-lg-0">
8     <li class="nav-item">
9       <a class="nav-link" href="#" @click.prevent="$emit('
navigate', 'Home')">Home</a>
10     </li>
11     <li v-if="userGrade" class="nav-item">
12       <a class="nav-link" href="#" @click.prevent="$emit('
navigate', 'DailyDiaryPage')">Diario Giornaliero</a>
13     </li>
14     <li v-if="userGrade === 'admin'" class="nav-item">
15       <a class="nav-link text-danger fw-bold" href="#"
@click.prevent="$emit('navigate', 'AdminProposalsPage')">
Gestisci Proposte</a>
16     </li>
17   </ul>
18 </template>

```

Listing 2: Emissione dei flussi di navigazione condizionale all'interno di NavBar.vue.

Il componente `App.vue` intercetta la notifica (`@navigate="setPage"`), muta reattivamente il valore di `currentPage` e attiva le direttive condizionali `v-if` nel template. Vue provvede così a smontare istantaneamente il componente obsoleto dal DOM e a iniettare la nuova vista in tempo reale, garantendo la fluidità tipica di un software desktop nativo.

5.1.2 Analisi di una Schermata Strutturale ed Informativa: HomeContainer.vue

Il componente `HomeContainer.vue` incarna il concetto di vista prevalentemente strutturale all'interno dell'applicazione. Il suo obiettivo ingegneristico è fungere da dashboard statica e punto di smistamento per le funzionalità operative dell'utente. Il layout è definito da elementi grafici predeterminati (*Action Cards*) che non modificano la propria topologia in base agli input dell'utente.

L'unico elemento di variabilità dinamica è rappresentato dal modulo di raccomandazione nutrizionale quotidiana. All'interno dell'hook del ciclo di vita `onMounted`, il componente esegue una chiamata HTTP asincrona di tipo `GET` tramite la libreria *Axios*, interrogando l'endpoint dedicato del backend per ricavare l'alimento consigliato dal sistema:

```
1 // HomeContainer.vue (<script setup>)
2 import axios from "axios"
3 import { onMounted, ref } from "vue"
4
5 const food = ref({})
6
7 const getTopFood = async () => {
8   try {
9     const response = await axios.get("http://localhost:3000/
10    foods/top-calorie")
11     food.value = response.data
12   } catch (e) {
13     console.error("Errore nel recupero dell'alimento
14     consigliato:", e)
15   }
16 }
17
18 onMounted(getTopFood)
```

Listing 3: Inizializzazione asincrona dei dati statici nel ciclo `onMounted` di `HomeContainer.vue`.

Una volta risolta la *Promise*, il riferimento reattivo `food` viene popolato, innescando l'aggiornamento automatico della View. Questo comportamento evidenzia l'efficienza del pattern MVVM, dove lo sviluppatore è sollevato dall'onere di dover ricostruire l'albero HTML, limitandosi a gestire lo stato logico del dato.

5.1.3 Analisi di una Schermata Dinamica e Data-Driven: DailyDiaryPage.vue

All'estremo opposto del livello di presentazione si colloca `DailyDiaryPage.vue`, un modulo ad altissima densità dinamica che rappresenta il cuore operativo dell'applicazione lato client. Questo componente deve orchestrare simultanea-

mente flussi CRUD complessi, mutazioni in tempo reale di array locali e calcoli predittivi basati sullo stato antropometrico dell'utente loggato.

Le principali scelte implementative che caratterizzano questo componente dinamico includono:

- **Sincronizzazione Concorrente dello Stato:** All'atto del montaggio, la pagina effettua richieste asincrone parallele per scaricare sia il catalogo alimentare globale, sia i record dei consumi già registrati nel database per la giornata corrente, popolando i vettori reattivi locali.
- **Filtrazione Semantica Client-Side:** Per evitare un sovraccarico di query sul database e azzerare i tempi di latenza, la ricerca dei cibi è implementata tramite una proprietà computata (`computed`). Vue monitora la stringa inserita dall'utente e applica un filtro in tempo reale sull'array in memoria, isolando i caratteri coincidenti senza generare traffico di rete.
- **Gestione dello Stato di Approvazione (Security Boundary):** L'interfaccia implementa una logica di sbarramento basata sul campo booleano `approved` presente all'interno dei documenti dei cibi. Quando un utente comune effettua una ricerca nel diario, il ViewModel si assicura di escludere dalla visualizzazione tutti i record che presentano il flag impostato a `false`. Questi ultimi rappresentano infatti alimenti proposti dalla community e non ancora verificati. Al contrario, tale limitazione decade se il ruolo rilevato in sessione è `admin`, consentendo l'accesso alla sezione di moderazione ed approvazione delle proposte pendenti.
- **Calcolo Predittivo dei Target Energetici:** La pagina estrae i parametri fisiologici correnti dell'utente (*peso, altezza, età, sesso e livello di attività*) dal `localStorage` ed esegue localmente l'algoritmo matematico per stabilire il target calorico personalizzato.

L'algoritmo predittivo di **Mifflin-St Jeor** viene integrato nel blocco logico del componente, risultando deputato alla determinazione del dispendio energetico giornaliero totale (TDEE):

```
1 // DailyDiaryPage.vue (Estrazione logica TDEE)
2 const getTDEE = () => {
3   const weight = parseFloat(localStorage.getItem('weight'))
4     || 0;
5   const height = parseFloat(localStorage.getItem('height'))
6     || 0;
7   const age = parseInt(localStorage.getItem('age')) || 0;
8   const gender = localStorage.getItem('gender') || 'M';
9   const activityLevel = localStorage.getItem('activityLevel')
10     || 'moderate';

  if (weight === 0 || height === 0 || age === 0) return
    2000; // Fallback di sicurezza
```

```

11 // Formula di Mifflin-St Jeor per la stima del metabolismo
    basale (BMR)
12 let bmr = (10 * weight) + (6.25 * height) - (5 * age);
13 bmr = (gender === 'M') ? bmr + 5 : bmr - 161;
14
15 // Coefficienti PAL (Physical Activity Level) per l'
    integrazione del TDEE
16 const multipliers = {
17   sedentary: 1.2,
18   moderate: 1.55,
19   very: 1.725,
20   extra: 1.9
21 };
22
23 return Math.round(bmr * (multipliers[activityLevel] ||
    1.2));
24 };
25
26 const maintenanceCalories = ref(getTDEE());

```

Listing 4: Algoritmo di calcolo del TDEE (Mifflin-St Jeor) integrato nel modulo di `DailyDiaryPage.vue`.

L'output numerico di `maintenanceCalories` viene infine legato in modo reattivo alla View. Qualora la sommatoria delle calorie introdotte mediante l'inserimento dei cibi superi la soglia del TDEE calcolato, il framework applica istantaneamente classi CSS condizionali (`:class`) agli indicatori grafici. L'interfaccia muta cromaticamente virando verso tonalità di avviso (rosso), fornendo un feedback visivo immediato di sforamento energetico senza richiedere cicli di osservazione o chiamate di verifica al server.

5.2 Backend

Il livello logico-funzionale e di elaborazione dati dell'applicazione è interamente affidato a un runtime Node.js operante in sinergia con il framework Express.js. Sul versante server, l'architettura adotta una declinazione rigorosa del pattern **MVC (Model-View-Controller)**, isolando i canali di persistenza, le regole di business logic e i punti di accesso API. Questo approccio garantisce l'estendibilità del codice, semplifica le procedure di manutenzione e disaccoppia le componenti server dalle logiche reattive del client.

5.2.1 Punto di Ingresso del Server e Separazione dei Ruoli (`index.js`)

L'orchestrazione iniziale del server è governata dal file `index.js`, il quale funge da punto di ingresso unificato per l'applicazione. Invece di centralizzare la computazione in un singolo script monolitico, il sistema implementa una scomposizione architetturale nota come **Separation of Concerns** (Separazione degli Interessi), articolata su tre macro-livelli:

- **Routes (Livello di Smistamento):** Moduli dedicati all'intercettazione dei vettori di richiesta HTTP (GET, POST, PUT, DELETE). Il loro unico scopo ingegneristico è mappare gli URL dell'API RESTful e instradare i payload in ingresso verso i rispettivi moduli logici.
- **Controllers (Livello Logico):** Componenti in cui risiede l'effettiva *Business Logic* dell'applicazione. Ricevono i dati validati dalle rotte, eseguono le computazioni matematiche o algoritmiche richieste e formulano le risposte HTTP strutturate in formato JSON da restituire al frontend.
- **Models (Livello di Persistenza):** Schemi definiti tramite l'ODM *Mongoose* che vincolano la topologia dei dati sul database MongoDB. Agiscono da sbarramento logico per garantire l'integrità del database prima di effettuare mutazioni fisiche sui documenti.

Il file `index.js` si occupa di inizializzare la connessione al database NoSQL, istanziare la pipeline dei middleware globali (quali la gestione del *Cross-Origin Resource Sharing - CORS* e il parsing automatico dei payload JSON) e registrare in modo modulare i router applicativi:

```

1 // index.js
2 require('dotenv').config();
3 const express = require('express');
4 const mongoose = require('mongoose');
5 const cors = require('cors');
6
7 // Connessione al database tramite indirizzamento locale
8 mongoose.connect('mongodb://127.0.0.1:27017/CoachCalories')
9   .then(() => console.log('MongoDB Connesso'))
10  .catch(err => console.error('Errore MongoDB:', err));
11
12 const app = express();
13
14 // Definizione ed esecuzione della pipeline dei middleware
15 // globali
16 app.use(cors());
17 app.use(express.json());
18 app.use(express.static('public'));
19
20 // Mappatura e segmentazione modulare delle rotte REST
21 app.use('/foods', require('./src/routes/foodsRoutes'));
22 app.use('/api/auth', require('./src/routes/authRoutes'));
23 app.use('/api/diary', require('./src/routes/diaryRoutes'));
24 app.use('/api/notifications', require('./src/routes/
25   notificationRoutes'));
26
27 app.listen(3000, () => {
28   console.log('Server unificato attivo sulla porta 3000');
29 });

```

Listing 5: Inizializzazione dei moduli, middleware logici e mappatura dei router in index.js.

Grazie a questa topologia, la scalabilità del backend è preservata: l'aggiunta di nuove entità nel database o l'espansione di rotte applicative non richiede la modifica dei core-middleware di sistema, ma si limita all'iniezione di file isolati all'interno delle rispettive cartelle di competenza.

5.2.2 Ingegnierizzazione della Business Logic e Calcolo Nutrizionale Proporzionale (diaryController.js)

Il nucleo computazionale dell'applicazione risiede nei controller deputati alla manipolazione del diario alimentare giornaliero. Quando un utente seleziona un alimento dal catalogo e ne altera la quantità standard rispetto a quella predefinita sul DB (es. consumando 180g di un alimento censito su base 100g), il backend non può accettare passivamente un inserimento arbitrario, poiché rischierebbe di inquinare l'integrità analitica delle statistiche.

La soluzione implementata all'interno del modulo `diaryController.js` prevede un algoritmo di ricalcolo proporzionale transazionale. All'atto della ricezione di un payload di inserimento, il controller estrae i parametri base dell'alimento certificato, determina un **fattore di scala matematico** (espresso dal rapporto tra la quantità desiderata e la quantità nativa) e rimappa dinamicamente l'apporto calorico e i macronutrienti (carboidrati, proteine, grassi) arrotondandone i valori per prevenire la proliferazione di cifre decimali infinite:

```
1 // src/controllers/diaryController.js
2 const Diary = require('../models/DiaryModel');
3
4 exports.addFoodToDiary = async (req, res) => {
5   try {
6     const { date, food, username, customQuantita } = req
7       .body;
8
9     if (!username) return res.status(400).json({ message
10       : 'Username non fornito' });
11
12     let diary = await Diary.findOne({ date, username });
13
14     // Determinazione del fattore di scala basato sulla
15     // proporzione matematica
16     const baseQuantita = Number(food.quantita) || 100;
17     const targetQuantita = customQuantita ? Number(
18       customQuantita) : baseQuantita;
19     const fattore = targetQuantita / baseQuantita;
20
21     // Istanziamento del sotto-documento ricalcolato in
22     // modo deterministico
```

```

18     const newFoodItem = {
19         foodId: food._id || food.foodId,
20         nome: food.nome,
21         calorie: Math.round((food.calorie || 0) *
fattore),
22         carboidrati_g: Number(((food.carboidrati_g || 0)
* fattore).toFixed(1)),
23         proteine_g: Number(((food.proteine_g || 0) *
fattore).toFixed(1)),
24         grassi_g: Number(((food.grassi_g || 0) * fattore
).toFixed(1)),
25         quantita: targetQuantita,
26         unita: food.unita || 'g'
27     };
28
29     if (!diary) {
30         // Inizializzazione della giornata alimentare
qualora non esistesse a DB
31         diary = new Diary({
32             date, username, foods: [newFoodItem],
33             totals: {
34                 calorie: newFoodItem.calorie,
35                 carboidrati_g: newFoodItem.carboidrati_g
36             },
37             proteine_g: newFoodItem.proteine_g,
38             grassi_g: newFoodItem.grassi_g
39         });
40     } else {
41         // Aggiornamento per append e rinfresco
incrementale dei totali macro-nutrizionali
42         diary.foods.push(newFoodItem);
43         diary.totals.calorie = Math.round((diary.totals.
calorie || 0) + newFoodItem.calorie);
44         diary.totals.carboidrati_g = Number(((diary.
totals.carboidrati_g || 0) + newFoodItem.carboidrati_g).
toFixed(1));
45         diary.totals.proteine_g = Number(((diary.totals.
proteine_g || 0) + newFoodItem.proteine_g).toFixed(1));
46         diary.totals.grassi_g = Number(((diary.totals.
grassi_g || 0) + newFoodItem.grassi_g).toFixed(1));
47     }
48
49     await diary.save();
50     return res.status(200).json(diary);
51 } catch (error) {
52     return res.status(500).json({ message: "Errore
interno del server" });
53 }
54 };

```

Listing 6: Logica di calcolo proporzionale e rinfresco transazionale dei totali in `diaryController.js`.

L'algoritmo applica una mutazione atomica sul documento della giornata. Nel caso in cui il diario di quella specifica data esista già, l'alimento proporzionato viene aggiunto in coda all'array dei consumi e i totali della giornata vengono incrementati in tempo reale. Questo garantisce che la risposta JSON inviata al client rifletta istantaneamente lo stato matematico aggiornato dell'apporto energetico.

5.2.3 Infrastruttura di Inferenza Semantica e Logica di Tool Calling (`chatbotController.js`)

La funzionalità architetturale più complessa del backend è l'integrazione di un motore di comprensione del linguaggio naturale (*NLP*) per consentire la compilazione automatica dei pasti tramite messaggi testuali a testo libero. Tale comportamento è implementato nel modulo `chatbotController.js`, il quale interfaccia il server con l'SDK di Groq per sfruttare i tempi di risposta ad altissima velocità garantiti dalle unità hardware LPU.

L'ingegnerizzazione di questo modulo non si limita all'inoltro del testo al modello *Llama 3.3*, ma sfrutta il paradigma avanzato del **Tool Calling**. Il backend inietta all'interno della chiamata API uno schema di funzioni predefinite (`tools`), descritte dettagliatamente tramite formati JSON Schema (es. `inserisci_cibo_oggi` o `autocompila_giorno_vuoto`). Il modello linguistico analizza la semantica del prompt dell'utente e, qualora rilevi un'intenzione operativa, inibisce la generazione di testo libero e risponde compilando un oggetto strutturato contenente gli argomenti isolati dal testo (es. nome del cibo estratto e grammatura stimata).

Al fine di garantire la stabilità di esecuzione ed evitare crash derivanti da risposte de-strutturate o imperfezioni del modello, il controller implementa un algoritmo di resilienza strutturato su cicli di tentativi alternati e disattivazione condizionale degli strumenti:

```
1 // src/controllers/chatbotController.js
2 const Groq = require('groq-sdk');
3 const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });
4
5 exports.handleChatMessage = async (req, res) => {
6   try {
7     const { message, username, chatHistory } = req.body;
8     if (!message) return res.status(400).json({ error: "
  Input mancante" });
9
10    // Analisi preventiva per disattivare i tool in caso
    di query puramente informative
11    let toolChoice = "auto";
12    const msgLower = message.toLowerCase();
```

```

13     const hasAction = ["mangiato", "bevuto", "aggiungi",
14       "inserisci", "riempi"].some(v => msgLower.includes(v));
15     if (!hasAction) toolChoice = "none"; // Inibisce i
16     tool se l'utente sta solo chiacchierando
17
18     const apiMessages = [{ role: "system", content: "
19     Prompt di istruzione e vincoli per l'LLM..." }];
20     apiMessages.push({ role: "user", content: message })
21     ;
22
23     let responseMessage = null;
24     let tentativi = 0;
25     let successo = false;
26
27     // Loop di resilienza per intercettare ed eliminare
28     JSON malformati dall'LLM
29     while (tentativi < 3 && !successo) {
30       tentativi++;
31       try {
32         const chatCompletion = await groq.chat.
33         completions.create({
34           messages: apiMessages, model: "llama
35           -3.3-70b-versatile", tools: tools, tool_choice:
36           toolChoice
37         });
38         responseMessage = chatCompletion.choices[0].
39         message;
40
41         // Validazione strutturale preventiva del
42         payload di risposta
43         if (responseMessage.tool_calls &&
44         responseMessage.tool_calls.length > 0) {
45           JSON.parse(responseMessage.tool_calls
46           [0].function.arguments);
47         }
48         successo = true; // Se il parsing non
49         solleva eccezioni, arresta il ciclo
50       } catch (err) {
51         if (tentativi === 3) {
52           return res.json({ reply: "Errore nella
53           decodifica dei numeri. Riprova.", action: "none" });
54         }
55       }
56     }
57
58     // Analisi ed esecuzione dello strumento invocato
59     dall'IA
60     if (responseMessage.tool_calls && responseMessage.
61     tool_calls.length > 0) {
62       let messaggiRisposta = [];

```



```

47         for (const toolCall of responseMessage.
48         tool_calls) {
49             const args = JSON.parse(toolCall.function.
arguments);
50
51             if (toolCall.function.name === "
52             inserisci_cibo_oggi") {
53                 // Esecuzione logica: inserisce nel DB l
'alimento con i macro estratti dall'IA
54                 // ed effettua il check incrociato con i
cibi certificati se presenti
55                 messaggiRisposta.push('Aggiunto **${args
56                 .nome_alimento}** (${args.calorie} kcal) al diario. ');
57             }
58             return res.json({ reply: messaggiRisposta.join
59             ('\n'), action: "food_inserted" });
60
61             // Restituzione della risposta testuale se non sono
stati invocati strumenti
62             return res.json({ reply: responseMessage.content,
63             action: "none" });
64         } catch (error) {
65             res.status(500).json({ error: "Errore di
elaborazione della chat." });
66         }
67     };

```

Listing 7: Ciclo di inferenza con Tool Calling, validazione del formato JSON e gestione delle eccezioni in chatbotController.js.

Il controller agisce come un perimetro di sicurezza logica: decodifica gli argomenti JSON passati dall'intelligenza artificiale, verifica la presenza dell'alimento all'interno della collezione dei cibi approvati per riallineare i macro in caso di match e aggiorna in autonomia la collezione dei diari su MongoDB. Questo meccanismo mappa richieste espresse in linguaggio naturale direttamente in record strutturati sul database relazionale, astruendo interamente la complessità del tracciamento per l'utente finale.

6 Verifica e test

La fase di verifica valida i requisiti dell'applicazione, garantendo la portabilità dell'ambiente isolato, la segregazione multi-utente e la tolleranza ai guasti del modulo di intelligenza artificiale.

6.1 Ambiente di Esecuzione e Seeding dei Dati

La persistenza dei dati è isolata in un container Docker MongoDB. Il popolamento iniziale (*seeding*) avviene in modalità incrementale tramite l'importazione di file in formato NDJSON. Sfruttando la protezione nativa dell'indice primario sull'identificativo unico del documento, la procedura inserisce in coda i record inediti e scarta automaticamente le righe che presentano una chiave duplicata, preservando l'integrità dei test precedenti e garantendo la portabilità del database.

```
1 # Caricamento e popolamento delle quattro collezioni core
2 docker cp users.json db-mongo-1:/tmp/users.json
3 docker exec -it db-mongo-1 mongoimport --db CoachCalories --
  collection users --file /tmp/users.json
4
5 docker cp foods.json db-mongo-1:/tmp/foods.json
6 docker exec -it db-mongo-1 mongoimport --db CoachCalories --
  collection foods --file /tmp/foods.json
7
8 docker cp diaries.json db-mongo-1:/tmp/diaries.json
9 docker exec -it db-mongo-1 mongoimport --db CoachCalories --
  collection diaries --file /tmp/diaries.json
10
11 docker cp notifications.json db-mongo-1:/tmp/notifications.
  json
12 docker exec -it db-mongo-1 mongoimport --db CoachCalories --
  collection notifications --file /tmp/notifications.json
```

Listing 8: Comandi Docker per il caricamento incrementale del database di test.

6.2 Campagna di Alpha Testing e Valutazione dell'Esperienza Utente

Al fine di analizzare il comportamento del software in un contesto d'uso quotidiano e prolungato, è stata condotta una campagna di Alpha Testing che ha coinvolto un gruppo di controllo composto da 5 utenti. I tester sono stati selezionati all'interno di una cerchia relazionale ristretta per agevolare il monitoraggio empirico delle sessioni. Dal punto di vista del target, i soggetti coinvolti non presentavano un interesse pregresso o specialistico verso il tracciamento nutrizionale rigido, configurando un quadro di utilizzo spontaneo e non vincolato, ideale per misurare l'attrito d'uso della piattaforma.

Sotto il profilo puramente estetico e d'interfaccia, l'applicazione ha riscosso un gradimento visivo unanime grazie alla pulizia del layout scuro e alla leggibilità dei componenti Bootstrap. I suggerimenti e le metriche di usabilità emersi durante i test sono stati analizzati e ripartiti in due categorie in base alla loro fattibilità architeturale.

6.2.1 Migliorie Implementate nel Sistema

Le modifiche ritenute prioritarie per ottimizzare l'efficienza dei flussi operativi e l'accessibilità dei dati sono state integrate direttamente nel codice sorgente del client:

1. **Centralizzazione dei Parametri Antropometrici:** È stata rilevata la necessità di poter consultare in modo immediato lo stato fisico corrente del profilo. La richiesta è stata soddisfatta mediante lo sviluppo del componente `PhysiologicalDataPage.vue`, il quale isola e presenta in una dashboard dedicata lo storico antropometrico persistito in sessione.
2. **Routine di Svuotamento dei Form di Inserimento:** Durante l'aggiunta consecutiva di più alimenti o di proposte di cibo, l'interfaccia manteneva i dati digitati nel payload precedente, rallentando l'esperienza d'uso. Il problema è stato risolto ingegnerizzando il metodo asincrono `resetForm()` all'interno dei moduli di sottomissione, il quale azzerava lo stato reattivo e ripulisce il DOM subito dopo la risoluzione della richiesta HTTP, permettendo inserimenti sequenziali immediati.

6.2.2 Suggerimenti Differiti o Respinti

I restanti contributi sono stati rinviati a futuri cicli di sviluppo o respinti per preservare la solidità dei vincoli architetturali originari, secondo le seguenti motivazioni tecniche:

1. **Suddivisione Cronologica dei Pasti (Colazione, Pranzo, Cena):** La frammentazione della giornata alimentare è stata esclusa poiché del tutto superflua ai fini del calcolo energetico matematico basato sul TDEE. L'introduzione di tali categorie avrebbe inutilmente appesantito la struttura a sotto-documenti della collezione dei diari su MongoDB; l'implementazione è stata tuttavia catalogata come potenziale feature di visualizzazione opzionale per il futuro.
2. **Enfatizzazione Visiva del Click di Selezione Alimenti:** L'interazione grafica corrente è stata ritenuta già conforme agli standard di feedback visivo grazie alle transizioni di hovering CSS sui componenti card. Una variazione più marcata non è apparsa strettamente necessaria in questa iterazione, pur restando aperta a ottimizzazioni successive.
3. **Ristrutturazione Radicale del Catalogo per la Modalità Admin:** È stata proposta la creazione di un'interfaccia catalogo completamente differente per gli amministratori. La scelta progettuale è stata quella di respingere l'istanza per preservare l'uniformità stilistica e la consistenza della User Experience (*UX Consistency*) cross-ruolo, limitando le differenze alla sola comparsa selettiva dei pulsanti di modifica e cancellazione basati sui permessi.

4. **Classificazione Rigida dei Cibi in Categorie Alimentari:** L'introduzione di tassonomie fisse (es. frutta, verdura, carboidrati) avrebbe generato forti ambiguità interpretative e arbitrarietà nella categorizzazione dei piatti composti. Inoltre, la ristrutturazione del backend avrebbe richiesto tempistiche incompatibili con i vincoli di rilascio del progetto, spingendo a rimandare la valutazione di un sistema di filtraggio categoriale a successive espansioni.

6.3 Test di Resilienza del Modulo NLP e Sanificazione dell'Input

L'ultima fase ha testato la robustezza del chatbot integrato con l'SDK di Groq e il modello *Llama 3.3* di fronte a tre scenari di input non nominali emersi in fase di validazione:

- **Input Conversazionali:** Se l'utente inserisce frasi informative o di cortesia senza intenzioni di tracciamento, il controller rileva l'assenza di verbi d'azione e imposta `tool_choice = "none"`, disattivando i tool per rispondere in puro testo.
- **Sanificazione dei Caratteri Speciali (Vulnerabilità NoSQL Injection):** L'inserimento di stringhe contenenti caratteri speciali (es. *"Yogurt greco 0%"* o stringhe con `&`, `$` e `#`) rappresenta un rischio critico sia per la stabilità sintattica del parser JSON dell'LLM, sia per la sicurezza dei dati. Il sistema intercetta preventivamente l'input applicando filtri di sanificazione tramite espressioni regolari. Questa pulizia a monte neutralizza potenziali tentativi di *NoSQL Injection* o alterazioni dei comandi inviati all'AI, stabilizzando il flusso.
- **Payload JSON Malformati e Fallback:** In caso di anomalie strutturali residue nel codice generato dall'LLM, il metodo `JSON.parse` solleva un'eccezione. Il backend intercetta l'errore ed esegue un loop di riprovazione (*While-Loop Retry*) fino a 3 tentativi. Se il fallimento persiste, il blocco `catch` attiva una *graceful degradation*, restituendo un messaggio di cortesia standardizzato senza compromettere la stabilità del runtime Node.js.

7 Deployment

Il rilascio e la configurazione di *CoachCalories* sono stati ottimizzati per consentire un'esecuzione rapida in ambiente locale, isolando le sole credenziali strettamente confidenziali per proteggere l'accesso ai servizi esterni di intelligenza artificiale.

7.1 Configurazione delle Variabili d'Ambiente (.env)

Per evitare il tracciamento di chiavi private nei sistemi di controllo versione e garantire la sicurezza dei token di accesso, l'applicazione si affida a un file `.env` memorizzato nella radice del backend. Nel contesto specifico del progetto, l'unica variabile d'ambiente esternalizzata è la chiave di autorizzazione per l'infrastruttura di inferenza, come illustrato di seguito:

```
1 # Chiave crittografica di autenticazione per l'SDK di Groq (
    Llama 3.3)
2 GROQ_API_KEY=gsk_vostro_token_segreto_api_groq
```

Listing 9: Struttura del file `.env` per l'isolamento delle credenziali del chatbot.

Il modulo `dotenv` intercetta questo file all'avvio del server Express e inietta il valore nel runtime di Node.js, rendendolo accessibile all'interno di `chatbotController.js` tramite la variabile globale `process.env.GROQ_API_KEY`.

Al contrario, per semplificare le procedure di deployment ed esecuzione in ambiente di sviluppo locale, parametri quali la porta del server (impostata sulla porta standard 3000) e l'indirizzo di connessione unificato al database MongoDB (`mongodb://127.0.0.1:27017/CoachCalories`) sono stati cablati direttamente (*hardcoded*) all'interno del file di ingresso `index.js`. Questa scelta progettuale riduce la complessità di configurazione iniziale senza pregiudicare la stabilità delle comunicazioni client-server.

7.2 Messa in Funzione Locale

L'installazione e l'esecuzione dell'ecosistema richiedono l'attivazione parallela dei due livelli disaccoppiati dell'applicazione, operanti secondo un'architettura client-server:

- **Inizializzazione del Backend Express.js:** Richiede il caricamento preliminare dei moduli lato server mediante il gestore di pacchetti Node (`npm install`) e il successivo avvio del servizio. All'inizializzazione, lo script `index.js` apre la pipeline dei middleware globali, stabilisce la connessione con il container MongoDB locale e valida la presenza della chiave Groq prima di mettersi in ascolto delle richieste HTTP.
- **Esecuzione del Frontend Vite.js:** Sul versante client, l'esecuzione del server di sviluppo locale gestisce il routing di stato reattivo e indirizza le richieste asincrone della libreria Axios verso la porta 3000 del backend, completando l'architettura operativa della piattaforma.

8 Conclusioni e sviluppi futuri

Il ciclo di progettazione, sviluppo e verifica di *CoachCalories* ha permesso di trarre con successo gli obiettivi architetturali prefissati, concretizzando una Single Page Application solida, reattiva e pienamente fruibile. Il sistema si dimostra uno strumento efficace e stabile per la gestione deterministica del bilancio energetico quotidiano, capace di astrarre la complessità del calcolo nutrizionale a beneficio dell'utente finale. La validità dell'infrastruttura è stata confermata anche da sessioni di auto-validazione empirica sul campo, durante le quali la piattaforma è stata impiegata con successo come applicazione di tracciamento reale per il monitoraggio prolungato di un piano alimentare personalizzato.

Tuttavia, per consentire una transizione sicura dall'attuale ambiente di staging locale a un deployment definitivo in produzione su rete pubblica (cloud open-to-web), emergono alcuni requisiti strutturali imprescindibili sul fronte dell'onboarding e della protezione dei dati:

- Modulo di Registrazione Nativo: Attualmente il sistema presuppone un popolamento controllato o amministrativo delle anagrafiche. È necessaria l'integrazione di una pagina di registrazione utente dotata di filtri di validazione dei campi in tempo reale lato client e controlli di sbarramento per l'unicità delle chiavi (*username* ed *email*) lato server.
- Inasprimento della Sicurezza sul Livello di Auth: Il backend richiede l'implementazione di politiche di sicurezza rigide per la gestione delle sessioni. Questo include la cifratura asimmetrica o l'hashing forte delle password tramite algoritmi standard (es. *Bcrypt*) prima della persistenza su MongoDB, l'adozione di protocolli di rinnovo dei token (Refresh Token) e l'introduzione di middleware per la prevenzione di attacchi di tipo Cross-Site Request Forgery (CSRF).

Oltre ai requisiti di sicurezza, l'analisi dei feedback raccolti durante la campagna di Alpha Testing ha tracciato una roadmap chiara per incrementare l'appetibilità commerciale e la granularità analitica della piattaforma attraverso tre principali direttrici di sviluppo futuro:

- Flessibilità nella Segmentazione dei Pasti: Sebbene l'accorpamento in un unico totale giornaliero sia matematicamente impeccabile ai fini del TDEE, l'introduzione di una visualizzazione opzionale o di un filtraggio condizionale per categorie temporali (colazione, pranzo, cena e spuntini) risponderebbe alle abitudini comportamentali di una vasta fascia di utenti, migliorando la fidelizzazione (*user engagement*).

- **Espansione delle Metriche Micro-Nutrizionali:** Per elevare il software da contatore calorico a strumento di monitoraggio salutistico avanzato, si prevede l'estensione degli schemi del database per censire e tracciare ulteriori indicatori critici quali grassi saturi, zuccheri liberi, sodio e fibre, offrendo una panoramica clinica completa del profilo nutrizionale.
- **Evoluzione e Potenziamento del Modulo di Intelligenza Artificiale:** Il nucleo basato su intelligenza artificiale presenta ampi margini di espansione. Una prima evoluzione prevede l'integrazione di modelli di *Computer Vision* in grado di analizzare e riconoscere la composizione macro-nutrizionale dei pasti partendo direttamente da fotografie scattate dall'utente. Inoltre, per superare i vincoli operativi delle API commerciali (quali limitazioni sul traffico, costi di scalabilità e latenza di rete), l'architettura futura prevede la dismissione dei servizi esterni in favore dell'addestramento o del *fine-tuning* di un Large Language Model open-source proprietario, ospitato localmente su infrastruttura hardware dedicata, garantendo così totale indipendenza tecnologica e una maggiore privacy nell'elaborazione dei dati sensibili degli utenti.